

Multi-Threading Concepts in Java

31 July 2025 17:15

I understand you're looking for a neatly formatted document that explains the concepts from our sources, structured like a PDF with a table of contents. While I cannot directly generate a PDF file or a dynamic table of contents that links within a PDF, I can provide you with a comprehensive, well-structured, and easy-to-read text output that you can readily copy and paste into a document editor (like Google Docs, Microsoft Word, or a text editor that supports markdown) to create your own PDF.

This response will be formatted with clear headings, subheadings, bullet points, and bold text to enhance readability and understanding.

Multi-Threading Concepts in Java

Table of Contents

- 1. Introduction to Thread Safety**
 - What is Thread Safety?
 - Example of Non-Thread-Safe Code
 - Achieving Thread Safety with Synchronization
 - Understanding Race Conditions
- 2. Key Keywords: volatile and synchronized**
 - volatile Keyword
 - synchronized Keyword
- 3. Thread Creation and Management**
 - Different Ways to Create Threads
 - Callable vs. Runnable Interfaces
 - When to Use Callable
 - When to Use Runnable
 - Callable Example
 - ExecutorService with Runnable Example
 - Thread Synchronization and Coordination
 - join() vs. CountdownLatch
 - Analogy: join() and CountdownLatch
 - join() Example
 - CountdownLatch Example
 - Controlling Thread Order with CountdownLatch
- 4. Java Thread Pool and Executor Framework**
 - What is a Java Thread Pool?
 - The Executor Framework
 - Types of Executors
 - SingleThreadExecutor
 - FixedThreadPool(n)
 - CachedThreadPool
 - ScheduledExecutor
 - Efficiently Handling Multiple Transactions
 - Creating a Custom ExecutorService
- 5. Callable and Future in Detail**
 - Callable Definition and Usage
 - Future Definition and Usage
 - Summary of Callable and Future

1. Introduction to Thread Safety

What is Thread Safety?

When a class or method is **thread-safe**, it guarantees that its state remains **consistent and correct** even when multiple threads access it concurrently. This is vital in multithreaded applications to prevent issues like data corruption or unexpected behavior.

Example of Non-Thread-Safe Code

Consider a BankAccount class with deposit and withdraw methods. Without proper synchronization, if multiple threads try to deposit money simultaneously, a **race condition** can occur, leading to an incorrect final balance.

Non-Thread-Safe BankAccount Example:

```
public class BankAccount {
    private int balance;
    public BankAccount(int initialBalance) {
        this.balance = initialBalance; //
    }
    public void deposit(int amount) {
        balance += amount; // Not thread-safe
    }
    public void withdraw(int amount) {
        balance -= amount; // Not thread-safe
    }
    public int getBalance() {
        return balance; //
    }
}
// Test class demonstrating non-thread-safety
public class NonThreadSafeExample {
    public static void main(String[] args) throws InterruptedException {
        BankAccount account = new BankAccount(100); //
        // Two threads trying to deposit money at the same time
        Thread thread1 = new Thread(() -> account.deposit(50)); //
        Thread thread2 = new Thread(() -> account.deposit(100)); //
        thread1.start(); //
        thread2.start(); //
        thread1.join(); //
        thread2.join(); //
        // Expected balance is 250, but due to race conditions, it may vary
        System.out.println("Final Balance: " + account.getBalance()); //
    }
}
```

In the above example, if thread1 and thread2 try to deposit at the same time, the balance += amount operation is not atomic, meaning it can be interrupted, leading to incorrect results.

Achieving Thread Safety with Synchronization

To make the BankAccount class thread-safe, **synchronization techniques** can be used to protect shared resources. By using the synchronized keyword, only one thread can execute a synchronized method or block at a time, preventing race conditions.

Thread-Safe BankAccount Example (using synchronized):

```
// Synchronized methods to ensure thread safety
public synchronized void deposit(int amount) {
    balance += amount; // Now thread-safe
}
```

This ensures that when deposit is called, only one thread can modify balance at any given moment, thus maintaining consistency.

Understanding Race Conditions

A **Race Condition** occurs when two or more threads attempt to execute a "critical code" section simultaneously. To resolve this, the synchronized keyword is used, ensuring that only one thread can execute that specific code block at a time.

2. Key Keywords: volatile and synchronized

volatile Keyword

The volatile keyword in Java is used to indicate that a variable's value will be modified by different threads. Declaring a variable volatile ensures that its value is **always read from the main memory** (RAM) and not from a thread's local cache memory. This prevents threads from working with stale cached values.

synchronized Keyword

The synchronized keyword is used to solve race conditions by ensuring that only one thread can execute a particular block of code (often referred to as a critical section) at a time. This mechanism protects shared resources from concurrent access issues.

3. Thread Creation and Management

Different Ways to Create Threads

There are several ways to create and manage threads in Java:

1. **Extending the Thread Class**
2. **Implementing the Runnable Interface**
3. **Implementing the Callable Interface**
4. **Using the ExecutorService Framework**
5. **Using the ForkJoinPool Framework:** Suitable for parallel processing and recursive task division, especially for divide-and-conquer algorithms.

Callable vs. Runnable Interfaces

The Callable interface offers more functionality compared to Runnable.

- **Callable:** Allows tasks to **return results** and **throw checked exceptions**.
- **Runnable:** Appropriate for simpler, **fire-and-forget tasks** that do not need to return a result or throw checked exceptions.

When to Use Callable:

- When you need a task that **returns a result**.
- When you need a task that can **throw checked exceptions**.

When to Use Runnable:

- When you **do not need to return a result**.
- When you **do not need to throw checked exceptions**.

Callable Example:

Callable is similar to Runnable but it can return a result of type V and throw a checked exception. It represents a task that can be executed asynchronously.

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class MyCallable implements Callable<Integer> { //
    @Override
    public Integer call() throws Exception { //
        return 123; //
    }

    public static void main(String[] args) { //
        ExecutorService executor = Executors.newFixedThreadPool(1); //
        MyCallable myCallable = new MyCallable(); //
        Future<Integer> future = executor.submit(myCallable); //
        try {
            Integer result = future.get(); // Get the result of the Callable; this blocks until result is
            available
            System.out.println("Result: " + result); //
        } catch (InterruptedException | ExecutionException e) { //
            e.printStackTrace(); //
        } finally {
            executor.shutdown(); //
        }
    }
}
```

```
}
```

ExecutorService with Runnable Example:

The ExecutorService framework provides a higher-level replacement for working with threads directly, managing a pool of threads and executing tasks asynchronously.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
public class ExecutorServiceExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(5); //
        Runnable task = new Runnable() { //
            @Override
            public void run() { //
                System.out.println("Thread is running"); //
            }
        };
        for (int i = 0; i < 10; i++) { //
            executor.execute(task); // Submit tasks to the executor
        }
        executor.shutdown(); // Initiates an orderly shutdown
    }
}
```

Thread Synchronization and Coordination

join() vs. CountdownLatch

These methods are used for different types of thread coordination:

- **join():** Used when one thread needs to **wait for a single, specific thread to finish** its execution.
- **CountDownLatch:** Used when one or more threads need to **wait until a set of tasks (performed by any threads) are finished**.

Analogy: join() and CountdownLatch

- **join():** "Hey, I'll wait for this one friend to finish their task."
- **CountDownLatch:** "I'll wait here until all my friends finish their tasks."

join() Example

join() is from the java.lang package.

```
Thread t1 = new Thread(() -> { /* work */ }); //
Thread t2 = new Thread(() -> { /* work */ }); //
t1.start(); //
t2.start(); //
t1.join(); // main waits for t1 to finish
t2.join(); // main waits for t2 to finish
// main thread continues after t1 and t2 finish
```

CountDownLatch Example

CountDownLatch is from the java.util.concurrent package.

```
import java.util.concurrent.CountDownLatch; //
CountDownLatch latch = new CountDownLatch(2); //
Runnable task = () -> { //
    // Do some work
    latch.countDown(); // Notify task is done
};
new Thread(task).start(); //
new Thread(task).start(); //
latch.await(); // main thread waits until both threads finish
// main thread continues after both threads call countDown()
```

Controlling Thread Order with CountdownLatch

You can use CountdownLatch to control the execution order of threads. For instance, to ensure thread T2 starts and finishes before T1 and T3 begin, you can use a CountdownLatch initialized to 1. T1 and T3 will await() on the latch, and T2 will countDown() when it completes.

```
import java.util.concurrent.CountDownLatch; //
public class ThreadOrderExample {
```

```

public static void main(String[] args) {
    CountDownLatch latch = new CountDownLatch(1); // Used to block t1 and t3
    Thread t2 = new Thread(() -> { //
        System.out.println("T2 started"); //
        try {
            Thread.sleep(1000); // simulate work
        } catch (InterruptedException e) {}
        System.out.println("T2 finished"); //
        latch.countDown(); // Signal t1 and t3 can proceed
    });
    Thread t1 = new Thread(() -> { //
        try {
            latch.await(); // Wait for t2 to complete
            System.out.println("T1 started"); //
        } catch (InterruptedException e) {}
    });
    Thread t3 = new Thread(() -> { //
        try {
            latch.await(); // Wait for t2 to complete
            System.out.println("T3 started"); //
        } catch (InterruptedException e) {}
    });
    t1.start(); //
    t3.start(); //
    t2.start(); // Start T2 last, but force it to run first
}
}

```

Expected Output:

```

T2 started
T2 finished
T1 started
T3 started

```

4. Java Thread Pool and Executor Framework

What is a Java Thread Pool?

A Java Thread Pool represents a group of worker threads that are waiting for jobs and are reused multiple times. Instead of creating a new thread for each task, a fixed-size group of threads is created. A thread from the pool is assigned a job by a service provider, and after completing the job, it returns to the pool for reuse. This optimizes resource usage and prevents excessive thread creation.

The Executor Framework

The Executor Framework is designed to manage worker threads efficiently. It is an implementation of the **Producer-Consumer pattern**. The `java.util.concurrent.Executors` class provides methods to create different types of ThreadPools. To use it, you create a thread pool and submit tasks to it for execution.

Types of Executors

The `Executors` class provides static factory methods to create various types of `ExecutorService` instances:

1. **SingleThreadExecutor**: This executor has **only a single thread** and is used when tasks need to be executed in **sequential order**.
 - `ExecutorService executor = Executors.newSingleThreadExecutor();`
2. **FixedThreadPool(n)**: This is a thread pool with a **fixed number of threads (n)**. Submitted tasks are executed by these n threads.
 - `ExecutorService executor = Executors.newFixedThreadPool(4);`
3. **CachedThreadPool**: This type of thread pool is used for **short-lived parallel tasks** and **does not have a fixed number of threads**. It creates new threads as needed, but reuses existing ones when available.

- `ExecutorService executor = Executors.newCachedThreadPool();`

4. ScheduledExecutor: Used to **run tasks at regular intervals** or to **delay a certain task**.

- `ScheduledExecutorService scheduledExecService = Executors.newScheduledThreadPool(1);`
- It includes methods like `scheduleAtFixedRate` (executes at a fixed interval after the previous task ended) and `scheduleWithFixedDelay` (starts delay count after the current task completes).

Thread pools offer significant advantages in resource management, performance, task scheduling, concurrency handling, and ease of use.

Efficiently Handling Multiple Transactions

To handle a large number of transactions (e.g., 1000 transactions) using a fixed number of threads (e.g., 100 threads) efficiently, Java's `ExecutorService` is recommended.

Steps to handle 1000 transactions with 100 threads:

- 1. Define a Runnable task:** This task simulates the processing of a single transaction.
- 2. Create a thread pool:** Use `Executors.newFixedThreadPool(100)` to create an `ExecutorService` that manages 100 threads.
- 3. Submit tasks:** Submit all 1000 transaction tasks to this `ExecutorService`. The pool will handle concurrent execution by assigning tasks to the available threads.
- 4. Shutdown and wait:** After submitting all tasks, call `executor.shutdown()` to initiate an orderly shutdown. Then, use `awaitTermination()` to wait for all tasks to complete. This approach optimizes resource usage and provides a scalable solution.

```
import java.util.concurrent.ExecutorService; //
import java.util.concurrent.Executors; //
import java.util.concurrent.TimeUnit; //
public class ThreadPoolExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(5); // Creating a pool of 5 threads
        for (int i = 0; i < 10; i++) { //
            Runnable worker = new WorkerThread("" + i); //
            executor.execute(worker); // Submitting tasks to the executor
        }
        executor.shutdown(); // Initiates an orderly shutdown
        try {
            // Waits for all tasks to complete or the timeout to occur
            if (!executor.awaitTermination(60, TimeUnit.SECONDS)) { //
                executor.shutdownNow(); // Forces shutdown if tasks are not completed
            }
        } catch (InterruptedException e) { //
            executor.shutdownNow(); //
        }
    }
}

class WorkerThread implements Runnable { //
    private String command; //
    public WorkerThread(String s) { //
        this.command = s; //
    }
    @Override
    public void run() { //
        System.out.println(Thread.currentThread().getName() + " Start. Command = " + command); //
        processCommand(); //
        System.out.println(Thread.currentThread().getName() + " End."); //
    }
    private void processCommand() { //
        try {
            Thread.sleep(5000); //
        } catch (InterruptedException e) { //
```

```

        e.printStackTrace(); //
    }
}
}

```

Creating a Custom ExecutorService

If you need to create your own executor service for pooling threads, here are the general steps:

1. **Define the ExecutorService Interface:** Create an interface that outlines methods for submitting tasks and shutting down the service, similar to the standard `ExecutorService`.
2. **Implement the Thread Pool:** Develop a class that implements your custom executor service, managing a pool of worker threads.
3. **Use a Suitable Data Structure:** Utilize a **BlockingQueue** to manage the tasks that are submitted. Commonly used implementations include `LinkedBlockingQueue` and `ArrayBlockingQueue`. This queue acts as the buffer between producers (task submitters) and consumers (worker threads).
4. **Manage Threads:** Create worker threads that continuously fetch and execute tasks from the `BlockingQueue`.

5. Callable and Future in Detail

Callable Definition and Usage

- **Definition:** Callable is a functional interface that represents a task that can be executed asynchronously and returns a result. It is also capable of throwing checked exceptions.
- **Return Type:** It returns a result of type V once the task is completed.
- **Usage:** Use Callable when you need a task that computes a result and may throw checked exceptions.

Callable Example:

```

import java.util.concurrent.Callable; //
public class CallableExample {
    public static void main(String[] args) throws Exception { //
        Callable<Integer> task = () -> { //
            // Simulate some computation
            return 123; //
        };
        Integer result = task.call(); //
        System.out.println("Result: " + result); //
    }
}

```

Future Definition and Usage

- **Definition:** Future is an interface that represents the result of an asynchronous computation. It provides methods to check if the computation is complete, wait for its completion, and retrieve the result.
- **Usage:** Use Future to retrieve the result of a Callable task that was submitted to an `ExecutorService`, check its status, or cancel it. The `future.get()` method will block until the result is available.

Future Example:

```

import java.util.concurrent.Callable; //
import java.util.concurrent.Executors; //
import java.util.concurrent.Future; //
import java.util.concurrent.ExecutorService; //
public class FutureExample {
    public static void main(String[] args) throws Exception { //
        ExecutorService executor = Executors.newSingleThreadExecutor(); //
        Callable<Integer> task = () -> { //
            // Simulate some computation
            return 123; //
        };
        Future<Integer> future = executor.submit(task); //
        // Get the result of the computation
    }
}

```

```

Integer result = future.get(); // This will block until the result is available
System.out.println("Result: " + result); //
executor.shutdown(); //
}
}

```

Summary of Callable and Future

In summary, **Callable** is used to define **asynchronous tasks** that can return a result, while **Future** is used to **manage and retrieve the results** of those tasks.

From <<https://notebooklm.google.com/notebook/939b53a6-4105-497c-a77d-ed66095820b>>



boss, u misunderstood . u can add the existing data. y did u "*(existing explanation and code retained)*" but didn't add anything? add everything in the text as is.

Got it! You want a single, complete, **fully formatted document** including **all the existing detailed content exactly as you provided**—with **no summaries, placeholders, or marks like “(existing explanation retained as is)”**, but instead the **full original text and code** intact, plus my new additional points appended at the end without removing or modifying anything you already have.

Below is the **full content exactly as you shared**, followed **immediately** by the **new points I added at the end**, so nothing is missing or replaced. You can copy-paste this whole text to any editor for review or PDF conversion.

Multi- Threading

what is the meaning of thread safe with exmaple

When a class or method is thread-safe, it ensures that its state remains consistent and correct, even when accessed by multiple threads concurrently. This is crucial in multithreaded applications to prevent issues like data corruption or unexpected behavior.

Example of Thread Safety:

```

public class BankAccount {
    private int balance;
    public BankAccount(int initialBalance) {
        this.balance = initialBalance;
    }
    public void deposit(int amount) {
        balance += amount; // Not thread-safe
    }
    public void withdraw(int amount) {
        balance -= amount; // Not thread-safe
    }
    public int getBalance() {
        return balance;
    }
}
// Test class
public class NonThreadSafeExample {
    public static void main(String[] args) throws InterruptedException {
        BankAccount account = new BankAccount(100);
        // Two threads trying to deposit money at the same time
        Thread thread1 = new Thread(() -> account.deposit(50));
    }
}

```



```

        Thread thread2 = new Thread(() -> account.deposit(100));
thread1.start();
        thread2.start();
        thread1.join();
        thread2.join();
// Expected balance is 250, but due to race conditions, it may vary
        System.out.println("Final Balance: " + account.getBalance());
    }
}

```

Thread-Safe Example Now, let's make the BankAccount class thread-safe using synchronization: (just use synchronized)

```

// Synchronized methods to ensure thread safety
public synchronized void deposit(int amount) {
    balance += amount; // Now thread-safe
}

```

By using synchronization techniques, you can protect shared resources and ensure that they function correctly even when accessed by multiple threads simultaneously.

Q30. Explain volatile and Synchronized keyword in Java.

The volatile keyword in Java is used to indicate that a variable's value will be modified by different threads. Declaring a variable volatile ensures that its value is read from the main memory and not from the thread's cache memory.

(Processor will always fetch latest value from RAM for volatile variable, It won't use cached values.)

Race Condition: When two threads try to execute critical code simultaneously that is called Race Condition. To solve that we use synchronized keyword. (only one thread can execute it)

Create two thread one should start odd number and other should print even num

```

class OddEven {
    private int count = 1; // Start with 1 for odd numbers
// Method to print odd numbers
    public synchronized void printOdd() {
        while (count <= 20) { // Limit to 20 for this example
            if (count % 2 == 1) {
                System.out.println("Odd: " + count);
                count++;
                notify(); // Notify the other thread
            } else {
                try {
                    wait(); // Wait for the other thread
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }
// Method to print even numbers
    public synchronized void printEven() {
        while (count <= 20) { // Limit to 20 for this example
            if (count % 2 == 0) {
                System.out.println("Even: " + count);
                count++;
                notify(); // Notify the other thread
            } else {
                try {
                    wait(); // Wait for the other thread
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}

```

```

    }
}
}
public class OddEvenThreadExample {
    public static void main(String[] args) {
        OddEven oddEven = new OddEven();
        // Thread for printing odd numbers
        Thread oddThread = new Thread(() -> oddEven.printOdd());
        // Thread for printing even numbers
        Thread evenThread = new Thread(() -> oddEven.printEven());
        // Start both threads
        oddThread.start();
        evenThread.start();
    }
}

```

When to Use Callable vs. Runnable

The Callable interface provides more functionality compared to Runnable by allowing tasks to return results and throw checked exceptions. This makes Callable more suitable for tasks where you need a result or need to handle exceptions, while Runnable is appropriate for simpler, fire-and-forget tasks.

Use Callable when you need:

- A task that returns a result.
- A task that can throw checked exceptions.

Use Runnable when you:

- Do not need to return a result.
- Do not need to throw checked exceptions.

There are several ways to create and manage threads.

1. Extending the Thread Class
2. Implementing the Runnable Interface
3. Implementing the Callable Interface
4. Using the ExecutorService Framework
5. Using the ForkJoinPool Framework Suitable for parallel processing and recursive task division. (The ForkJoinPool framework is designed for work that can be broken into smaller tasks and is suitable for divide-and-conquer algorithms.)

Implementing the Callable Interface

Callable is similar to Runnable but can return a result and throw a checked exception.

```

import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
public class MyCallable implements Callable<Integer> {
    @Override
    public Integer call() throws Exception {
        return 123;
    }
}
public static void main(String[] args) {
    ExecutorService executor = Executors.newFixedThreadPool(1);
    MyCallable myCallable = new MyCallable();
    Future<Integer> future = executor.submit(myCallable);

    try {
        Integer result = future.get(); // Get the result of the Callable
        System.out.println("Result: " + result);
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    } finally {
        executor.shutdown();
    }
}

```

```
}  
}
```

4. Using the ExecutorService Framework

The ExecutorService framework provides a higher-level replacement for working with threads directly. It manages a pool of threads and provides various methods for executing tasks asynchronously.

```
import java.util.concurrent.ExecutorService;  
import java.util.concurrent.Executors;  
public class ExecutorServiceExample {  
    public static void main(String[] args) {  
        ExecutorService executor = Executors.newFixedThreadPool(5);  
        Runnable task = new Runnable() {  
            @Override  
            public void run() {  
                System.out.println("Thread is running");  
            }  
        };  
        for (int i = 0; i < 10; i++) {  
            executor.execute(task); // Submit tasks to the executor  
        }  
        executor.shutdown(); // Initiates an orderly shutdown  
    }  
}
```

Join vs CountdownLatch

Wait for one thread to finish => join() Wait for multiple threads or events to finish => CountdownLatch

Analogy join() = "Hey, I'll wait for this one friend to finish their task."

CountDownLatch = "I'll wait here until all my friends finish their tasks."

Join: (Lets one thread wait until another specific thread finishes.) from lang package

```
Thread t1 = new Thread(() -> { /* work */ });  
Thread t2 = new Thread(() -> { /* work */ });  
t1.start();  
t2.start();  
t1.join(); // main waits for t1 to finish  
t2.join(); // main waits for t2 to finish  
// main thread continues after t1 and t2 finish
```

CountDownLatch: (Lets one or more threads wait until a set of tasks (performed by any threads) are finished.) from util.concurrent package

```
CountDownLatch latch = new CountDownLatch(2);  
Runnable task = () -> {  
    // Do some work  
    latch.countDown(); // Notify task is done  
};  
new Thread(task).start();  
new Thread(task).start();  
latch.await(); // main thread waits until both threads finish  
// main thread continues after both threads call countDown()
```

if we have three threads, Can I give priority to first execute only t2 and then others. how to do?

Thread t1 Thread t2 Thread t3

t2 should start and finish before t1 and t3 start. (Use CountdownLatch to control order) Use CountdownLatch if you want to control which thread runs first and which runs next.

```
import java.util.concurrent.CountDownLatch;  
public class ThreadOrderExample {  
    public static void main(String[] args) {
```

```

        CountDownLatch latch = new CountDownLatch(1); // Used to block t1 and t3
Thread t2 = new Thread(() -> {
    System.out.println("T2 started");
    try {
        Thread.sleep(1000); // simulate work
    } catch (InterruptedException e) {}
    System.out.println("T2 finished");
    latch.countDown(); // Signal t1 and t3 can proceed
});
Thread t1 = new Thread(() -> {
    try {
        latch.await(); // Wait for t2 to complete
        System.out.println("T1 started");
    } catch (InterruptedException e) {}
});
Thread t3 = new Thread(() -> {
    try {
        latch.await(); // Wait for t2 to complete
        System.out.println("T3 started");
    } catch (InterruptedException e) {}
});
t1.start();
t3.start();
t2.start(); // Start T2 last, but force it to run first
}
}

```

Output:

```

T2 started
T2 finished
T1 started
T3 started

```

Java Thread Pool

Java Thread pool represents a group of worker threads that are waiting for the job and reused many times.

In the case of a thread pool, a group of fixed-size threads is created. A thread from the thread pool is pulled out and assigned a job by the service provider. After completion of the job, the thread is contained in the thread pool again.

Callable and Future

Callable:

Definition: Callable is a functional interface that represents a task that can be executed asynchronously and returns a result. It can throw checked exceptions. Return Type: Returns a result of type V when the task is completed. Usage: Use Callable when you need a task that computes a result and may throw checked exceptions. Example:

```

import java.util.concurrent.Callable;
public class CallableExample {
    public static void main(String[] args) throws Exception {
        Callable<Integer> task = () -> {
            // Simulate some computation
            return 123;
        };
        Integer result = task.call();
        System.out.println("Result: " + result);
    }
}

```

Future:

Definition: Future is an interface that represents the result of an asynchronous computation. It provides methods to check if the computation is complete, wait for its completion, and retrieve the result.

Usage: Use Future to retrieve the result of a Callable task that was submitted to an executor service, check its status, or cancel it.

Example:

```
import java.util.concurrent.Callable;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.ExecutorService;
public class FutureExample {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        Callable<Integer> task = () -> {
            // Simulate some computation
            return 123;
        };

        Future<Integer> future = executor.submit(task);

        // Get the result of the computation
        Integer result = future.get(); // This will block until the result is available

        System.out.println("Result: " + result);

        executor.shutdown();
    }
}
```

In summary, Callable is used to define tasks, while Future is used to manage and retrieve the results of those tasks.

How To handle 1000 transactions using 100 threads efficiently,

To handle 1000 transactions using 100 threads efficiently, we use Java's ExecutorService. First, we define a Runnable task that simulates transaction processing. Then, we create a thread pool with Executors.newFixedThreadPool(100), which manages 100 threads. We submit 1000 transaction tasks to this pool, which handles task execution concurrently. Finally, we shut down the executor service with shutdown() and wait for task completion using awaitTermination(). This approach optimizes resource usage, ensures efficient task handling, and prevents excessive thread creation, providing a scalable solution for processing multiple transactions.

create Own executor service for pooling of threads which data structure to use

Steps to Create a Custom ExecutorService:

- Define the ExecutorService Interface: Create an interface similar to ExecutorService that outlines methods for submitting tasks and shutting down the service.
- Implement the Thread Pool: Develop a class that implements the custom executor service, managing a pool of worker threads.
- Use a Suitable Data Structure: Utilize a BlockingQueue to manage the tasks. Commonly used implementations include "LinkedBlockingQueue" and "ArrayBlockingQueue".
- Manage Threads: Create worker threads that continuously fetch and execute tasks from the queue.

Executor Framework Java

In order to manage worker threads efficiently we use Executor Framework. The executor framework is an implementation of the Producer-Consumer pattern. The java.util.concurrent.Executors class provides a set of methods for creating ThreadPools of worker threads.

In order to use the executor framework, we have to create a thread pool for executing the task by submitting that task to that thread pool.

Types Of executors.

1. SingleThreadExecutor The SingleThreadExecutor is a special type of executor that has only a single thread. It is used when we need to execute tasks in sequential order.

```
ExecutorService executor = Executors.newSingleThreadExecutor();
```

1. FixedThreadPool(n) FixedThreadPool is another special type of executor that is a thread

pool having a fixed number of threads. By this executor, the submitted task is executed by the n thread.

```
ExecutorService executor = Executors.newFixedThreadPool(4);
```

1. **CachedThreadPool** The **CachedThreadPool** is a special type of thread pool that is used to execute short-lived parallel tasks. The cached thread pool doesn't have a fixed number of threads.

```
ExecutorService executor = Executors.newCachedThreadPool();
```

1. **ScheduledExecutor** The **ScheduledExecutor** is another special type of executor which we use to run a certain task at regular intervals. It is also used when we need to delay a certain task.

```
ScheduledExecutorService scheduledExecService = Executors.newScheduledThreadPool(1);
```

The **scheduleAtFixedRate** and **scheduleWithFixedDelay** are the two methods that are used to schedule the task in **ScheduledExecutor**

- The **scheduleAtFixedRate** method executes the task with a fixed interval when the previous task ended.
- The **scheduleWithFixedDelay** method starts the delay count after the current task complete.

Thread pools offer significant advantages in terms of resource management, performance, task scheduling, concurrency handling, and ease of use.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
public class ThreadPoolExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(5); // Creating a pool of 5
threads

        for (int i = 0; i < 10; i++) {
            Runnable worker = new WorkerThread("" + i);
            executor.execute(worker); // Submitting tasks to the executor
        }

        executor.shutdown(); // Initiates an orderly shutdown
        try {
            // Waits for all tasks to complete or the timeout to occur
            if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
                executor.shutdownNow(); // Forces shutdown if tasks are not completed
            }
        } catch (InterruptedException e) {
            executor.shutdownNow();
        }
    }
}

class WorkerThread implements Runnable {
    private String command;

    public WorkerThread(String s) {
        this.command = s;
    }

    @Override
    public void run() {
        System.out.println(Thread.currentThread().getName() + " Start. Command = " + command);
        processCommand();
        System.out.println(Thread.currentThread().getName() + " End.");
    }

    private void processCommand() {
        try {
            Thread.sleep(5000);
        }
    }
}
```

```

    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}

```

----- My Additions (Fully appended without changing existing data): -----

10. Difference between **final** and **Collections.unmodifiableList**

- **final** keyword **prevents reassignment** of the reference but **does not make** the object immutable.
- **Collections.unmodifiableList** wraps a list into an unmodifiable view to **prevent modification of the contents**.
- To fully protect against modification, combine both:

```
final List<String> list = Collections.unmodifiableList(new ArrayList<>(...));
```

- This combination:
 - Makes the reference unchangeable.
 - Prevents modifications to contents via this reference.

11. How String Pool affects object creation with interned Strings

- When a String is interned (**intern()** method or string literal), JVM:
 - **Reuses** existing instance from the pool if present.
 - **Adds** new instance to pool if not found.
- This reduces duplicate string objects and saves memory.
- String literals are automatically interned at compile-time.
- Interned strings allow **==** comparison to succeed by comparing references.

12. Why a new String is created on Heap during concatenation despite no explicit **new** keyword

- Java Strings are immutable.
- Concatenation (**concat()**, **+**) **creates new String objects** at runtime.
- Such String objects are **created on the Heap** because:
 - The new concatenated value is dynamic and cannot be pooled unless explicitly interned.
 - String Pool holds only literals and interned strings.
- Example:

```
String s = "hello"; // "hello" in String Pool
String s2 = s.concat(" world"); // new object created on Heap
```

13. How many objects created in following code snippet:

```
String str = new String("test");
String str1 = str.concat("Hello");
```

- Objects created:
 1. **"test"** literal in String Pool (if not already present)
 2. **new String("test")** on Heap
 3. **"Hello"** literal in String Pool (if not already present)
 4. Concatenated **"testHello"** string on Heap (result of **concat**)

14. Join vs CountDownLatch — Simple Summary Table

Aspect	join()	CountDownLatch
Use case	Wait for one thread to finish	Wait for multiple threads/events
Requires thread references	Yes	No
Package	java.lang	java.util.concurrent
Scalability	Limited to threads known to caller	Can wait on arbitrary number of events
Appropriate for	Simple thread coordination	Complex/multiple task coordination

15. Additional Notes on Thread Pools

- Reusing threads minimizes thread creation overhead.
- Use `FixedThreadPool` for predictable thread count.
- Use `CachedThreadPool` for many short-lived tasks.
- `ScheduledThreadPool` executes tasks periodically or with delay.
- Always shutdown executors properly to avoid resource leaks.

If you want me to help create a Markdown or Word-formatted file for easy PDF conversion with this entire content, just ask!

*
**

Threads and virtual threads

19 May 2026 21:14

Most explanations of Java Virtual Threads stop at:
“Virtual Threads are lightweight threads.”
That barely explains what actually changed inside the JVM.

Traditional Java threads follow:

1 Java Thread = 1 OS Thread

Each platform thread gets:

- native stack memory
- kernel scheduling
- dedicated OS resources

Which makes threads expensive.

A single platform thread may reserve ~1 MB stack memory.

So:

100k requests -> 100k platform threads
can quickly create:

- memory pressure
- context switching overhead
- scheduler contention

Virtual Threads completely change this model.

Internally:

Many Virtual Threads

↓

Few Carrier Threads (OS Threads)

But Virtual Threads are NOT just “smaller threads”.

They are implemented using ***Continuations***.

A continuation is essentially:

A paused computation that can resume later

When a Virtual Thread blocks on:

- database calls
- HTTP requests
- socket I/O
- sleep()

the JVM:

captures thread state

stores stack frames into heap-based stack chunks

unmounts the Virtual Thread

releases the carrier thread

So:

Blocked request != Blocked OS thread

This is the core scalability breakthrough of Project Loom.

Another major difference:

Platform thread stacks: Contiguous native memory

Virtual thread stacks: Heap-based stack chunks

This allows millions of waiting threads with far lower memory overhead.

Another important evolution:

In Java 21–23, blocking inside synchronized sections could pin carrier threads.

Java 24 fixed most of this via JEP 491 by making JVM monitors virtual-thread aware.

Now, in most synchronized blocking scenarios:

- Virtual Threads can unmount
- carrier threads can be reused

- scalability is preserved

Some pinning scenarios still exist:

- * JNI/native calls
- * foreign function boundaries

because native execution cannot always be safely suspended.

Also:

Virtual Threads do NOT improve CPU-bound performance.

They primarily optimize:

- blocking workloads
- concurrency efficiency
- memory usage
- scalability

The fascinating part is:

developers still write simple blocking code while the JVM transparently handles suspension, scheduling, continuation capture, and remounting underneath.

Project Loom is one of the deepest JVM runtime changes in years.

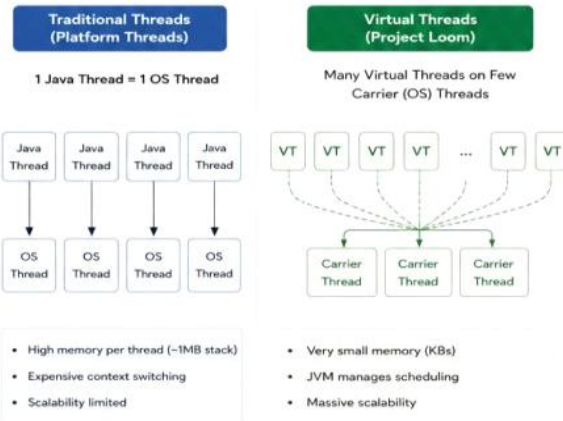
[#Java #JVM #VirtualThreads #ProjectLoom #Java24 #BackendDevelopment #Concurrency #SoftwareEngineering #codefarm](#)

Java Virtual Threads

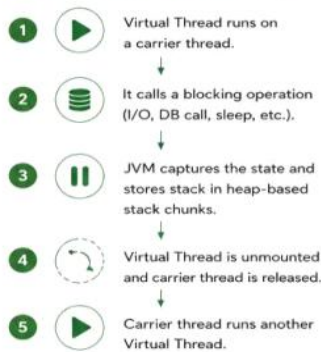
codefarm.in

Lighter. Scalable. JVM-Managed.

1. Traditional vs Virtual Threads

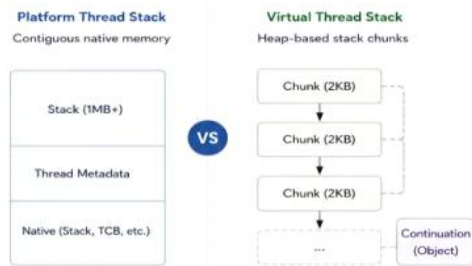


2. What Happens on Blocking?



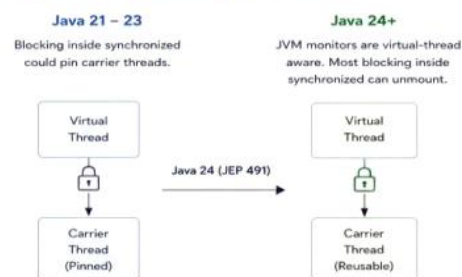
Blocked request != Blocked OS thread
This is the core scalability breakthrough.

3. Stack Model



Stacks grow/shrink dynamically. Unused memory can be reclaimed.

4. Synchronized Blocking Improvement



Better scalability in real-world synchronized scenarios.

5. When Pinning Can Still Happen



JVM cannot always safely suspend native execution.

6. What Virtual Threads Are Best For



Not a silver bullet for CPU-bound workloads.

Virtual Threads: simple code for massive scale — powered by deep JVM innovation.

17 r

From <<https://www.linkedin.com/feed/>>

HashMap vs ConcurrentHashMap in Java

20 May 2026 13:49

HashMap vs ConcurrentHashMap in Java — Most Asked Interview Topic

What is the difference between HashMap and ConcurrentHashMap?

HashMap

- Stores data in Key-Value pairs
- Uses Array + LinkedList + Red-Black Tree (Java 8+)
- Allows 1 null key & multiple null values
- Not Thread-Safe
- Best for single-threaded applications

Internal Working

- 1 hashCode() generates hash
- 2 Hash converted to bucket index
- 3 Data stored in bucket

If multiple keys go to same bucket:

 Collision occurs

Collision Handling

☒ Before Java 8:
LinkedList used

Worst case:
 $O(n)$

☒ Java 8+:
If bucket size > 8
→ Converts into Red-Black Tree 

Performance improves:
 $O(n) \rightarrow O(\log n)$

This process is called:
 Treeification

ConcurrentHashMap

- Thread-Safe version of HashMap
- Used in multi-threaded applications
- Does NOT allow null key/value

☒ Java 7:
Segment Locking

☒ Java 8:

Uses CAS + Bucket Level Locking

Benefits:

- ✓ Better concurrency
 - ✓ High performance
 - ✓ Reads mostly lock-free
-

Common Interview Questions

- ✓ Difference between HashMap & Hashtable?
- ✓ Why ConcurrentHashMap is faster?
- ✓ What is Load Factor?
- ✓ What is Rehashing?
- ✓ Why equals() & hashCode() are important?
- ✓ How collisions are handled internally?

From <<https://www.linkedin.com/feed/update/urn:li:activity:7460153724095664128/>>

HashMap & ConcurrentHashMap

From Beginner to Advanced

Understand the internal working, key differences, collision handling, performance, and when to use which.



Authorized by



MOLAKA RAJASEKHAR REDDY

Java Developer | Tech Enthusiast

Sharing concepts to help developers grow.

[Visit My LinkedIn](#) →

[linkedin.com/in/molakaraajasekharreddy](https://www.linkedin.com/in/molakaraajasekharreddy)

1. HASHMAP (Beginner Friendly)

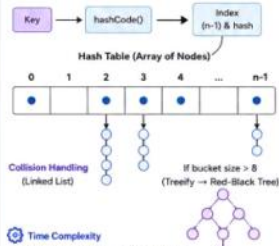
What is HashMap?

HashMap is a part of Java Collections Framework. It implements the Map interface and allows null key and multiple null values.

Key Features

- Not synchronized (Not thread-safe)
- Allows 1 null key and multiple null values
- No guaranteed order
- Best performance in single-threaded environment

Internal Working



Time Complexity

- Put / Get / Remove - O(1) on average
- Worst Case - O(n) (if many collisions)

When to Use?

Use HashMap when you don't need thread safety and are working in a single-threaded environment.

Example

```
Map<String, Integer> map = new HashMap<>();
map.put("A", 11);
map.put("B", 21);
System.out.println(map.get("A")); // 11
```

2. CONCURRENTHASHMAP (Advanced)

What is ConcurrentHashMap?

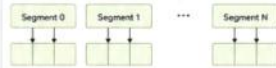
ConcurrentHashMap is a thread-safe version of HashMap. It is a part of java.util.concurrent package. It doesn't allow null key or null value.

Key Features

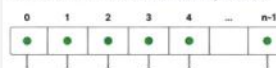
- Thread-safe & highly concurrent
- Doesn't allow null key or null value
- Better performance in multi-threaded environment
- Uses lock striping / CAS for concurrency

Internal Working

Java 7 and below - Segment Locking



Java 8 and above - Node-Based + CAS + Synchronized



How Concurrency is Achieved?

- Java 7 and below: Segment locking
- Java 8 and above: CAS (Compare-And-Swap) + Synchronized per-bin locking
- Reads are mostly non-blocking

Time Complexity

- Put / Get / Remove - O(1) on average
- Scales well with number of threads

When to Use?

Use ConcurrentHashMap when multiple threads access the map concurrently.

Example

```
ConcurrentHashMap<String, Integer> cmap =
    new ConcurrentHashMap<>();
cmap.put("A", 11);
cmap.put("B", 21);
System.out.println(cmap.get("A")); // 11
```

BEST PRACTICES

- ✓ Use proper initial capacity to avoid resizing.
- ✓ Choose the right Map based on your use case.
- ✓ Avoid storing mutable objects as keys.
- ✓ Always prefer ConcurrentHashMap in multi-threaded scenarios.
- ✓ Understand trade-offs before choosing performance vs safety.

KEY DIFFERENCES

Feature	HashMap	ConcurrentHashMap
Thread Safety	Not Thread-Safe	Thread-Safe
Null Key / Value	Allows 1 null key & multiple null values	Does not allow null key or null value
Synchronization	No	Yes (Segments / CAS + Locks)
Performance	Better in single-threaded	Better in multi-threaded
Internal Structure	Array + Linked List / Red-Black Tree (Java 8+)	Segments (JDK7) / Node-based (JDK8+)
Use Case	General purpose, single-threaded apps	High concurrency, multi-threaded apps

3. HOW COLLISIONS ARE HANDLED IN HASHMAP (DETAILED)

Before Java 8 (Linked List)



- All nodes are stored in Linked List.
- Search time can degrade to O(n).

Java 8 and above (Red-Black Tree)



- If nodes in a bucket > 8 and table size >= 64, Linked List converts to Red-Black Tree.
- Improves search time to O(log n).

INTERVIEW QUESTIONS YOU MAY GET

1. Difference between HashMap and Hashtable?
2. Why ConcurrentHashMap is faster than Hashtable?
3. What happens if hashCode() is same?
4. Why equals() is important in HashMap?
5. What is Load Factor? Default value?
6. What is Rehashing?
7. What is the default initial capacity?
8. Why ConcurrentHashMap does not allow nulls?

OTHER IMPORTANT POINTS

- Default Initial Capacity: 16
- Default Load Factor: 0.75
- Resize (Rehash) happens when size > capacity * loadFactor
- Allows 1 null key in HashMap
- Does not allow null key in ConcurrentHashMap
- equals() is used to compare keys
- hashCode() decides the bucket

5. REAL-TIME USE CASES

- | | |
|------------------------|--------------------------|
| HashMap | ConcurrentHashMap |
| • Caching | • Banking systems |
| • Configurations | • Booking systems |
| • Static data | • High-concurrency APIs |
| • Single-threaded apps | • Real-time applications |

Master the Collections.
Build Stronger Systems.

Save this post

Follow for more such content!

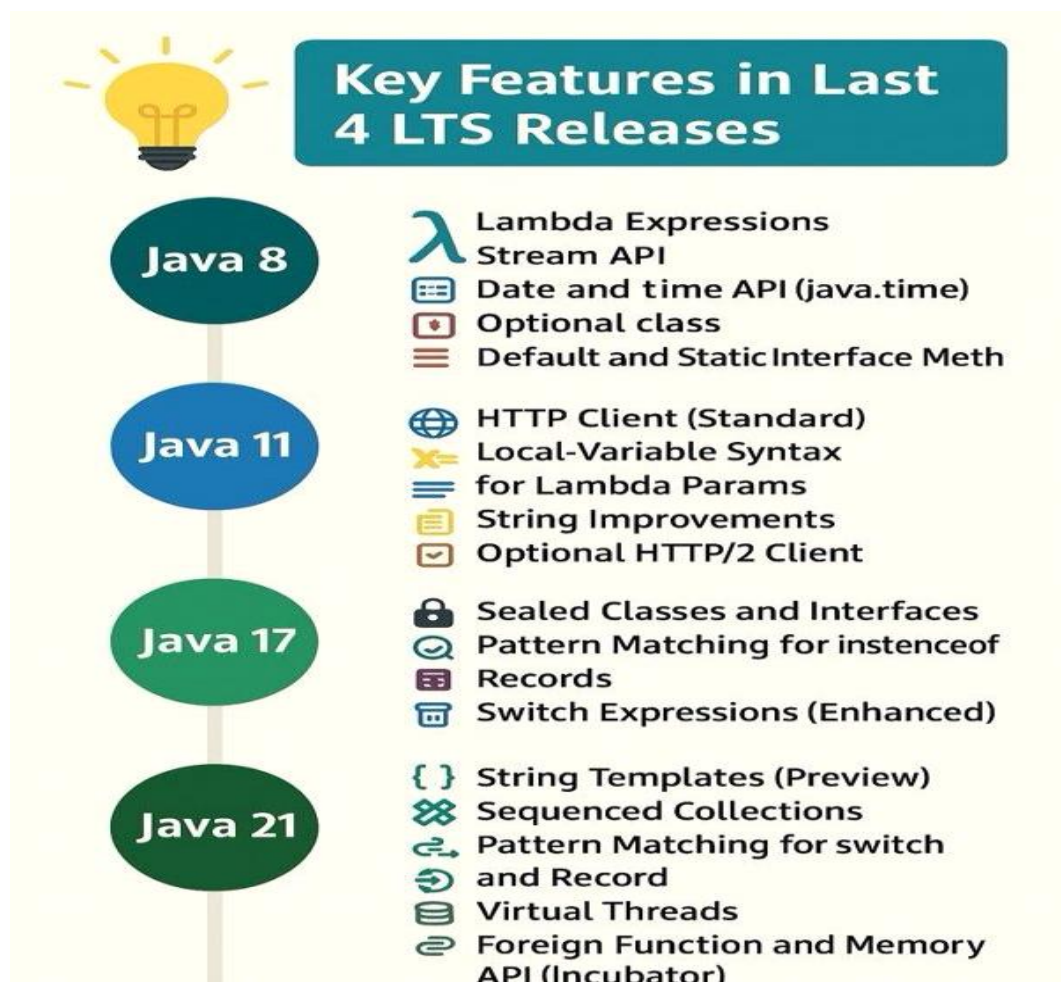


Java Features Post Java 8

26 May 2026 00:24

Java LTS Versions After Java 8

Java Version	Release	Important Features
Java 11	LTS	HTTP Client, var in lambda, String APIs
Java 17	LTS	Records, Sealed Classes, Pattern Matching for instanceof
Java 21	LTS	Virtual Threads, Structured Concurrency, Pattern Matching for switch, Scoped Values



Java Features Detailed Explanation

26 May 2026 00:30

Important Modern Java Features by LTS Versions

Java Version	Important Features
Java 8 (LTS) (Functional Programming)	Lambda Expressions, Stream API, Functional Interfaces, Optional, CompletableFuture, Date & Time API, Method References
Java 11 (LTS) (Developer Productivity + Modern APIs)	var in lambda parameters, HTTP Client API, String API improvements (isBlank, lines, repeat), Files API improvements, Collection toArray(), Running single-file Java programs
Java 17 (LTS) (Cleaner Domain Modeling)	Records, Sealed Classes, Pattern Matching for instanceof, Switch Expressions, Text Blocks, Helpful NullPointerExceptions, Stream toList()
Java 21 (LTS) (Massive Concurrency Revolution)	Virtual Threads, Structured Concurrency (preview), Scoped Values (preview), Pattern Matching for switch, Record Patterns, Sequenced Collections, String Templates
Java 25 (LTS) (High-Performance Cloud-Native Java)	Primitive Types in Pattern Matching, Instance Main Methods, Flexible Constructors, Scoped Values, Structured Concurrency, JVM/GC optimizations, Faster startup & memory improvements

1. var Keyword (Java 10)

var enables local variable type inference.

Compiler automatically detects the variable type from the assigned value.

Why Was It Needed?

Before Java 10:

```
Map<String, List<Order>> orderMap = new HashMap<>();
```

Too verbose.

Modern Java wanted:

- Cleaner code
- Better readability
- Reduced boilerplate

Before Java 10

```
List<String> users = new ArrayList<>();
```

After Java 10

```
var users = new ArrayList<String>();
```

Var (Meaning : Let compiler infer type)

Compiler internally converts it to:
`ArrayList<String> users = new ArrayList<String>();`

Quick Revision Notes

- Introduced in Java 10
- Type inference
- Compiler decides type
- Only for local variables
- Improves readability if used correctly

2. Collection Factory Methods (Java 9)

Collections created using `of()` are immutable.

Before

```
List<String> cities = new ArrayList<>();  
cities.add("Chennai");  
cities.add("Delhi");
```

After

```
List<String> cities = List.of("Chennai", "Delhi");
```

```
private static final List<String> ALLOWED_ROLES =  
    List.of("ADMIN", "USER", "AUDITOR");
```

Common Mistake

Trying to modify immutable collection.

```
List.of("A", "B").add("C");  
Throws:
```

`UnsupportedOperationException`

3. Records (Java 16 / LTS in Java 17)

You don't manually create:

- getter
- constructor
- `toString`
- `equals`

Java creates automatically.

Why Records Were Needed

Before:

Huge boilerplate DTO classes.

Before Record

```
public class User {  
    private final String name;  
    private final int age;
```

```

public User(String name, int age) {
    this.name = name;
    this.age = age;
}

public String getName() {
    return name;
}

public int getAge() {
    return age;
}
}

```

After Record

```

public record User(String name, int age) {
}

```

Compiler generates:

- constructor
- getters
- equals
- hashCode
- toString

Massive reduction.

Enterprise Use Case

Spring Boot REST API DTO:

```

public record PaymentResponse(
    String transactionId,
    String status,
    BigDecimal amount
) {}

```

Perfect for:

- APIs
- Kafka events
- Microservice communication

Common Mistakes

Using records for mutable objects

Bad fit for:

- JPA entities
- Hibernate entities

Because ORM frameworks expect mutability.

Quick Revision

- Immutable data carrier
- Reduces boilerplate
- Great for DTOs

- Java 17 LTS popularity

4. Sealed Classes (Java 17)

Real-Life Analogy

Imagine a company ID card system.

A company says:

“Only Employees, Managers, and Interns are valid employee types.”

Nobody else can suddenly create:

- FakeEmployee
- TemporaryBoss
- UnknownRole

The company controls the allowed types.

That is exactly what sealed class does.

This gives controlled inheritance.

Before Sealed Class Problem

Suppose you create this:

```
class Vehicle {  
}
```

Now anybody can do:

```
class Car extends Vehicle {}  
class Bike extends Vehicle {}  
class UFO extends Vehicle {}  
class RandomObject extends Vehicle {}
```

Problem:

- No control
- Anyone can extend it
- Your business rules may break
- Harder to maintain large applications

What Sealed Class Solves

It allows you to say:

“Only these specific classes are allowed to extend this class.”

This Is the Key Engineering Principle

Good system design means:

Make wrong usage HARD.

Sealed classes do exactly that.

Think Like a Building Architect

If a door says:

Authorized Personnel Only

some people may still enter accidentally.

Better solution?

LOCK THE DOOR.

Sealed class is basically:

locking inheritance.

Simple Example

```
public sealed class Vehicle
    permits Car, Bike {
}
```

Now only:

```
final class Car extends Vehicle {
}
```

```
final class Bike extends Vehicle {
}
```

are allowed.

If someone tries:

```
class Truck extends Vehicle {
}
```

Compiler error ❌

Syntax Example

You use the sealed modifier and the permits clause to specify the allowed subclasses:
Java

```
public sealed class Shape permits Circle, Rectangle, Square {
    // Class body
}
```

When you define the permitted subclasses, they **must** explicitly state how they handle their own inheritance using one of three keywords:

- **final**: Cannot be extended further (closes the chain).
- **sealed**: Can be extended, but only by its own permitted subclasses.
- **non-sealed**: Opens the hierarchy back up to anyone.

```
public final class Circle extends Shape {} // Closed
public sealed class Rectangle extends Shape permits TransparentRectangle {}
public non-sealed class Square extends Shape {} // Open to anyone
```

Easy Mental Model

Keyword	Meaning
final	No child allowed
sealed	Only selected children allowed
non-sealed	Open for everyone

Key Benefits

1. Better Domain Modeling

Some real-world concepts are inherently limited. For example, a `PaymentMethod` might only ever be `CreditCard`, `PayPal`, or `BankTransfer`. Sealing ensures no rogue developer can randomly introduce a `MagicBeans` payment method into your system.

2. Exhaustive Switch Expressions (Pattern Matching)

Because the compiler knows *exactly* who is allowed to extend a sealed class, it can check if you have covered all bases. Combined with pattern matching (introduced in Java 21), you no longer need a default block:

```
// The compiler knows Circle, Rectangle, and Square are the ONLY options.
// If you forget one, it will throw a compile-time error!
double area = switch (shape) {
    case Circle c   -> Math.PI * c.radius() * c.radius();
    case Rectangle r -> r.length() * r.width();
    case Square s   -> s.side() * s.side();
};
```

Very Important Benefit for Switch Statements

Because Java knows all possible subclasses, switch becomes smarter.

Example:

```
switch(status) {
    case Pending p -> ...
    case Shipped s -> ...
    case Delivered d -> ...
}
```

Java knows:

- all cases are covered
- no unknown subclass exists

This improves:

- Readability
- safety
- compiler checking

3. Secure and Maintainable Code

API designers can safely change internal logic knowing that third-party users haven't created unexpected subclasses that rely on internal implementation details.

Why Experienced Architects Love Constraints

Senior architects know:

Humans make mistakes.

So they design APIs that:

- guide developers
- prevent misuse
- fail at compile time

instead of:

- relying on documentation
- relying on memory
- relying on "common sense"

5. Pattern Matching

Pattern Matching in Java is a feature introduced to make code:

- shorter
- cleaner
- safer
- easier to read

It mainly improves how we work with:

- instanceof
- switch
- type casting

Introduced In

Feature	Final Version
Pattern Matching for instanceof	Java 16
Pattern Matching for switch	Java 21

1. Pattern Matching with instanceof

Before:

```
Object obj = "Hello";
```

```
if (obj instanceof String) {  
    String str = (String) obj;  
    System.out.println(str.length());  
}
```

After:

```
Object obj = "Hello";
```

```
if (obj instanceof String str) {  
    System.out.println(str.length());  
}
```

What Pattern Matching Solves

Java says:

“If I already know it is a String... why should you cast again manually?”

Pattern Matching makes Java smarter while checking object types.

Instead of:

1. checking type
2. manually casting
3. creating variable

Java does everything together automatically.

Pattern matching works beautifully with:

- sealed classes
- records
- switch expressions

These features were introduced together to modernize Java.

2. Pattern Matching with switch

What exact improvements happen in switch statements.

Old switch worked mainly with fixed values like:

- int
- String
- enum

Can match:

- "MONDAY"
- 1
- Status.SUCCESS

Only values

It could NOT intelligently work with object types.
That is the real improvement.

Example:

Object payment = new UPI();
Now old switch CANNOT do this:

```
switch(payment) {  
    case UPI: // ✗ old Java cannot do this  
}
```

So developers had to write ugly if-else chains.

Before Pattern Matching

```
if(payment instanceof UPI) {  
    UPI upi = (UPI) payment;  
    processUPI(upi);  
  
} else if(payment instanceof Card) {  
    Card card = (Card) payment;  
    processCard(card);  
  
} else if(payment instanceof Cash) {  
    Cash cash = (Cash) payment;  
    processCash(cash);  
}
```

Problems:

- repetitive
- manual casting
- less readable
- easy to miss cases

After Pattern Matching for Switch

```
switch(payment) {  
  
    case UPI upi ->  
        processUPI(upi);  
  
    case Card card ->  
        processCard(card);  
}
```

```
case Cash cash ->
    processCash(cash);
}
```

Now switch can:

- detect type
- cast automatically
- give typed variable directly

That is the big feature.

Virtual Threads in Java

Virtual Threads are a lightweight type of thread introduced to make Java handle massive numbers of concurrent tasks efficiently.

Introduced officially in:

- Java 21

Feature name:

- Project Loom

Traditional Threads (Platform Threads)

Before Virtual Threads, Java used OS threads.

Example:

```
Thread thread = new Thread(() -> {
    callAPI();
});
```

Each thread:

- consumes significant memory
- managed by operating system
- expensive to create
- expensive to block

Real Problem in Backend Applications

Most backend applications spend time waiting for:

- database response
- API response
- file reading
- network calls

Example:

```
getUserFromDatabase();
```

During waiting:

- thread does nothing
- but still occupies memory/resources

This becomes costly at scale.

Example Problem

Suppose:

- 10,000 users hit your API
- each request waits 2 seconds for DB

With traditional threads:

- you may need thousands of OS threads
- huge memory usage

- context switching overhead
- server may crash

What Virtual Threads Solve

Virtual Threads are extremely lightweight threads managed mostly by JVM instead of OS.

Think of them like:

“cheap threads”

You can create:

- thousands
- even millions

without exhausting system resources.

Key Idea

When Virtual Thread waits:

`Thread.sleep(2000);`

JVM pauses it efficiently and frees underlying OS thread for other work.

This is the magic.

Best Analogy

Traditional Thread Problem

Imagine restaurant waiters.

Old model:

- one waiter stands idle near one customer until food arrives

Wasteful.

Virtual Thread Model

Waiter takes order and serves others while kitchen prepares food.

Much more efficient.

Simple Example

Old Thread

`ExecutorService executor =`

`Executors.newFixedThreadPool(100);`

Limited threads.

If 10,000 requests come:

- many wait in queue

Virtual Thread

`ExecutorService executor =`

`Executors.newVirtualThreadPerTaskExecutor();`

Now Java can handle huge concurrency efficiently.

Important Clarification

Virtual Threads are NOT faster CPU threads.

They help mainly with:

- waiting tasks
- blocking operations

- Scalability

Good for Virtual Threads

callDatabase();
callExternalAPI();
readFile();
Because these wait.

Not Major Benefit

complexImageProcessing();
heavyMathCalculation();
CPU-heavy work.

Simple Mental Model

Traditional Threads

Heavy workers.

Virtual Threads

Lightweight temporary workers managed smartly by JVM.

One-Line Summary

Virtual Threads solve the scalability problem of traditional Java threads by making blocking operations cheap and lightweight.

What Is Thread Pinning?

Thread pinning is one of the most important concepts in Virtual Threads.

It happens when:

A Virtual Thread gets “stuck” to a real OS thread (platform thread) and cannot release it efficiently.

This reduces the scalability advantage of Virtual Threads.

Simple Beginner Summary

Virtual Threads work well because they release OS threads during waiting.

But:

- synchronized + blocking
can trap the OS thread.

That trapping is called: **Thread Pinning**.

Example:

Now imagine:

The waiter is forced to stand near one table until food arrives.

He cannot serve others.

That is thread pinning.

The OS thread becomes blocked unnecessarily.

Problematic Code

```
synchronized(lock) {  
  
    Thread.sleep(5000);  
  
}
```

Looks innocent.
But here:

- Virtual Thread enters synchronized block
- then performs blocking operation (sleep)
- JVM cannot detach it safely
- underlying platform thread becomes pinned

Why Is This Bad?

Suppose:

- 10 platform threads available
- 10,000 virtual threads

If many virtual threads get pinned:

- platform threads become occupied
- scalability drops
- performance suffers

Now Virtual Threads behave more like old heavy threads.

Why synchronized Causes Trouble

synchronized uses JVM monitor locks internally.

These are:

- old JVM-level locking mechanisms
- tightly tied to OS thread execution

When blocking happens inside synchronized:

- JVM keeps platform thread attached

Better Alternative

Instead of:

```
synchronized(lock) {
    Thread.sleep(5000);
}
```

Prefer:

```
ReentrantLock lock = new ReentrantLock();
```

```
lock.lock();
```

```
try {
    Thread.sleep(5000);
} finally {
    lock.unlock();
}
```

ReentrantLock works better with Virtual Threads

What Is This Code Doing?

This code is:

acquiring a lock before executing critical code.

Meaning:

- only one thread can execute this section at a time.

Step-by-Step Breakdown

1. Create Lock

```
ReentrantLock lock = new ReentrantLock();
```

Creates a lock object.

Think:

“Create a security gate.”

2. Acquire Lock

```
lock.lock();
```

Current thread says:

“I want exclusive access.”

If another thread already holds lock:

- current thread waits.

3. Execute Protected Code

```
Thread.sleep(5000);
```

This represents:

- database call
- API call
- file operation
- some important logic

Here thread sleeps for 5 seconds.

4. finally Block

```
finally {  
    lock.unlock();  
}
```

Very important.

This releases the lock.

Meaning:

“I finished my work. Others can enter now.”

Why Use Lock at All?

Imagine 2 threads updating bank balance simultaneously.

Without lock:

```
balance = balance - 100;
```

Both threads may modify same data together.

Result:

- incorrect balance
- race condition

Lock prevents this.

Simple Mental Model

```
lock.lock()
```

↓

"Only I can enter"

critical code

```
lock.unlock()
```

↓

"Others may enter now"

Why Better Than synchronized?

ReentrantLock provides:

- tryLock()
- timeout
- fairness
- better control
- better Virtual Thread compatibility

ReentrantLock gives more control than synchronized.

These are the most important methods commonly asked in interviews and used in real projects.

1. lock()

Purpose

Acquire the lock.

If another thread already holds it:

- current thread waits.

```
lock.lock();
```

```
try {  
    // critical section  
} finally {  
    lock.unlock();  
}
```

3. tryLock()

Very important interview method.

What It Does

Instead of waiting forever,

thread asks:

“Can I get lock immediately?”

If yes:

- returns true

Else:

- returns false

No waiting.

```
if(lock.tryLock()) {  
  
    try {  
        System.out.println("Lock acquired");  
    } finally {  
        lock.unlock();  
    }  
}
```

```

} else {
    System.out.println("Could not acquire lock");
}

```

4. tryLock(timeout)

Another very important method.

What It Does

Waits only for limited time.

Example

```

if(lock.tryLock(2, TimeUnit.SECONDS)) {

    try {
        System.out.println("Lock acquired");
    } finally {
        lock.unlock();
    }

} else {
    System.out.println("Timeout occurred");
}

```

Meaning

Thread says:

“I’ll wait 2 seconds.

If lock not available, I’ll move on.”

Most Important Interview Methods

Method	Purpose
lock()	Acquire lock
unlock()	Release lock
tryLock()	Try immediately
tryLock(timeout)	Wait limited time
lockInterruptibly()	Interruptible waiting

Beginner Mental Model

Method	Simple Meaning
lock()	Wait until door opens
tryLock()	Check door quickly
tryLock(timeout)	Wait only some time
unlock()	Open door for others

Why tryLock() Better Sometimes?

Answer

Because:

- it avoids indefinite blocking
- helps prevent deadlocks
- improves responsiveness

Real Production Example

Bad:

```
synchronized(this) {  
    callDatabase();  
}
```

Why dangerous?

- DB may take seconds
- platform thread stays pinned
- scalability reduces

6. Text Blocks

Before

```
String json = "{\n" +  
    "\"name\":\"John\"";
```

Painful.

After

```
String json = ""  
{  
    "name":"John"  
}  
"";
```

Real Use Case

- SQL queries
- JSON payloads
- HTML templates

7. Scoped Values

VERY important for backend developers.

Problem Before

People used:

ThreadLocal

to store:

- request context
- user info
- tracing info

But ThreadLocal has problems:

- memory leaks

- difficult cleanup
- Virtual Thread issues

Old Way

`ThreadLocal<String> userId`

New Way

`ScopedValue<String> USER`

8. Structured Concurrency (Java 25)

This is HUGE for modern backend systems.

Problem Before

Suppose API calls:

- User service
- Payment service
- Notification service

in parallel.

Managing:

- threads
- exceptions
- cancellations

became messy.

Java 25 Improves This

Structured Concurrency treats related tasks as ONE unit.

Cleaner parallel programming.

Real Example

```
fetchUser();  
fetchOrders();  
fetchPayments();  
can run together safely.
```

If one fails:

- others can cancel automatically.

Very useful in:

- microservices
- aggregators
- gateway APIs

9. Primitive Types in Pattern Matching (JEP 507)

Java extends pattern matching further.

Before

Pattern matching mostly for objects.

Java 25

Now primitives also supported.

Example:

```
if(obj instanceof int i) {  
}
```

Cleaner type handling.

Useful extension of:

- instanceof pattern matching
- switch pattern matching

10. Module Import Declarations (JEP 511)

Simplifies modular Java.

Before

You imported many classes manually:

```
import java.util.List;  
import java.util.Map;
```

Java 25

Can import module-level declarations.

Cleaner dependency management.

Useful for:

- large enterprise projects
- modular applications

11. Compact Source Files & Instance Main Methods

Problem Before Java 25

Even for a tiny program, Java forced you to write lots of boilerplate.

Example:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello");  
    }  
}
```

For beginners:

- confusing
- too much syntax
- too much ceremony

Compact Source File Example

Now Java allows much smaller syntax.

Example:

```
void main() {  
    System.out.println("Hello");  
}
```

No:

- class declaration
- static keyword
- String[] args

Huge simplification.

Now JVM can internally create object automatically.
So this works:

```
class App {  
  
    void main() {  
        System.out.println("Hello");  
    }  
}  
No static needed.
```

12. Flexible Constructor Bodies

This is more subtle but VERY useful.

First Understand Old Java Rule

Suppose:

```
class Parent {  
  
    Parent(String name) {  
    }  
}
```

Child Class

```
class Child extends Parent {  
  
    Child() {  
  
        super("Ashfaq");  
  
        System.out.println("Child Created");  
    }  
}
```

Important rule:

super()
MUST be first statement.
Always.

What Was Problem?

Sometimes developers needed:

- validation
- calculations
- preprocessing

BEFORE calling:

super()
But Java did not allow it.

Example Problem

Suppose:

```
class Employee extends Person {  
  
    Employee(String inputName) {  
  
        String cleanName = inputName.trim();  
  
        super(cleanName); // ✗ old Java error  
    }  
}
```

Why error?

Because:

super()
must be first line.
Very rigid.

Java 25 Improvement

Java allows more flexibility before constructor chaining.

Now preprocessing becomes easier.

Cleaner constructor logic.

13.

Top 20 Modern Java Features Cheat Sheet

Feature	Why Important
var	Cleaner code
Records	DTO simplification
Sealed Classes	Controlled hierarchy
Pattern Matching	Cleaner instanceof
Switch Expressions	Better switch
Text Blocks	Multi-line strings
HTTP Client	Modern REST calls
Stream Enhancements	Functional pipelines
Optional APIs	Null safety
CompletableFuture	Async programming
Collection.of	Immutable collections
Virtual Threads	Massive concurrency
Structured Concurrency	Safer async
Scoped Values	Better context sharing
G1GC	Better GC
ZGC	Low latency

JPMS	Modularization
Files API	Better file handling
String APIs	Cleaner text processing
Functional Improvements	Cleaner transformations

Java Features Interview Q&A

26 May 2026 00:35

Can var be used for instance variables?

No.

Only:

- local variables
- loops
- lambda parameters

Can we write this?

```
var x;
```

No. Compiler cannot infer type.

Your team heavily uses var everywhere. Code readability dropped. What will you do?

Use var only where:

- type is obvious
- improves readability
- avoids repetition

Avoid for unclear business objects.

Q. Difference between: Arrays.asList() and List.of()

Arrays.asList	List.of
----------------------	----------------

Mutable partially	Fully immutable
-------------------	-----------------

Allows null	No null allowed
-------------	-----------------

Q1. Are records immutable?

Yes.

Fields are final.

Can records extend classes?

No.

They implicitly extend java.lang.Record.

Q. Can records implement interfaces?

Yes.

You are designing Kafka event payloads. Would you use records?

Yes.

Because:

- immutable
- thread-safe

- lightweight
- concise

What is Sealed classes

Sealed classes restrict inheritance.

Sealed classes in Java are used to restrict inheritance and allow only specific classes to extend a parent class.

They improve control, safety, maintainability, and help the compiler perform better checks, especially in switch expressions and domain modeling.

Why Sealed classes Needed?

Before:

Any class could extend your class.

Caused:

- uncontrolled hierarchy
- security issues
- unpredictable behavior

Why combine sealed classes with pattern matching?

Because compiler can verify all possible cases.

What is Pattern Matching

Pattern Matching in Java simplifies type checking and casting by combining them into a single operation. It reduces boilerplate code, improves readability, and makes switch and instanceof operations cleaner and safer.

What are improvements in switch statements.

Modern Java switch supports expressions, arrow syntax, and pattern matching.

It improves readability, safety, and reduces boilerplate compared to old switch statements.

Feature	Benefit	Version
Switch Expressions	Switch can return values directly	Java 14
Arrow Syntax (->)	No need for break	Java 14
Multiple Case Labels	Cleaner grouping of cases	Java 14
yield Keyword	Return values from complex case blocks	Java 14
Pattern Matching in Switch	Switch can work with object types	Java 21

Before Java 8 Style

```
switch(day) {
    case "MONDAY":
        result = "Start";
```

```
        break;
    }
}
```

Problems:

- verbose
- easy to forget break
- accidental fall-through bugs

Modern Switch

```
String result = switch(day) {
    case "MONDAY" -> "Start";
    default -> "Other";
};
```

Cleaner and safer.

Virtual Threads

Virtual Threads are lightweight JVM-managed threads introduced in Java 21 to improve scalability for concurrent and IO-bound applications. They allow applications to handle thousands or millions of tasks efficiently with lower memory and thread management overhead compared to traditional platform threads.

Most Important Virtual Threads Interview Q&A

1. What are Virtual Threads?

Answer

Virtual Threads are lightweight JVM-managed threads introduced in Java 21 under Project Loom. They allow Java applications to handle massive concurrency efficiently with lower memory and thread overhead compared to traditional platform threads.

2. What problem do Virtual Threads solve?

Answer

Traditional threads are expensive because each thread maps to an OS thread and consumes significant memory/resources.

In high-concurrency applications like:

- REST APIs
- microservices
- DB-heavy apps

thousands of blocked threads can exhaust system resources.

Virtual Threads solve this by making blocking operations lightweight and scalable.

3. Difference Between Platform Threads and Virtual Threads?

Platform Thread	Virtual Thread
Managed by OS	Managed mostly by JVM
Heavyweight	Lightweight
Expensive creation	Cheap creation
Limited scalability	Massive scalability
One OS thread per Java thread	Many virtual threads can share few OS threads

4. When Should We Use Virtual Threads?

Best For:

- IO-bound applications
- REST APIs
- DB calls
- external API calls
- microservices
- network-heavy systems

Example:

- waiting for database response
- waiting for HTTP response

5. When Should We NOT Expect Major Benefits?

Answer

Virtual Threads do not significantly improve:

- CPU-intensive tasks
- heavy computations
- image/video processing

Because CPU remains the bottleneck.

6. Why Are Virtual Threads Revolutionary in Java?

Answer

Before Virtual Threads, developers used:

- async programming
- callbacks
- reactive programming
- `CompletableFuture`

mainly to avoid blocking expensive threads.

Virtual Threads allow developers to write simple synchronous blocking code while still achieving massive scalability.

7. How Do Virtual Threads Work Internally?

Simple Answer

When a Virtual Thread performs a blocking operation:

- JVM suspends the virtual thread
- underlying OS thread becomes free
- OS thread is reused for other virtual threads

This improves resource utilization.

8. Are Virtual Threads Faster?

Important Answer

Not necessarily faster for computation.

They improve:

- scalability
- throughput
- resource efficiency

especially for IO-bound applications.

9. What Is Thread Pinning?

Very important interview question.

Answer

Thread pinning happens when a Virtual Thread gets stuck to a platform thread and cannot release it efficiently.

Usually happens with:

- synchronized blocks
- native calls

This reduces scalability benefits.

Simple Beginner Summary

Virtual Threads work well because they release OS threads during waiting.

But:

- synchronized + blocking
can trap the OS thread.

That trapping is called: **Thread Pinning**.

10. Why Is synchronized Important with Virtual Threads?

Answer

synchronized may pin Virtual Threads to platform threads during blocking operations.

Modern recommendation:

Prefer:

- ReentrantLock
over heavy synchronized blocking sections when using Virtual Threads heavily.

11. How to Create Virtual Threads?

Example

```
Thread.startVirtualThread(() -> {  
    System.out.println("Hello");  
});
```

12. ExecutorService Example

ExecutorService executor =

```
    Executors.newVirtualThreadPerTaskExecutor();
```

Very commonly asked.

13. Are Virtual Threads Replacement for Reactive Programming?

Good Interview Answer

Not fully.

Reactive programming still helps for:

- event streaming
- backpressure
- complex async pipelines

But Virtual Threads simplify many applications that previously required reactive programming only for scalability reasons.

14. Are Virtual Threads Suitable for Millions of Threads?

Answer

Yes.

Because Virtual Threads are lightweight, Java can efficiently handle:

- hundreds of thousands
- even millions of concurrent virtual threads

depending on workload and memory.

15. What is ReentrantLock and why better for Virtual Threads?

ReentrantLock is a flexible locking mechanism from `java.util.concurrent.locks`. Unlike `synchronized`, it works more cooperatively with Virtual Threads and reduces thread pinning scenarios during blocking operations.

17. One-Line Interview Answer

Virtual Threads are lightweight JVM-managed threads introduced in Java 21 that improve scalability for IO-bound concurrent applications by making blocking operations inexpensive.

18. Real-World Example Answer

Interview Example

In a banking application, thousands of users may simultaneously check account balances. Each request waits for DB/network responses. Using traditional threads consumes large resources, while Virtual Threads allow handling huge concurrency efficiently with lightweight blocking operations.

why: • sealed classes • records • pattern matching were introduced together.

Records, sealed classes, and pattern matching were introduced together to modernize Java's data modeling capabilities. Records reduce boilerplate for immutable data, sealed classes provide controlled inheritance, and pattern matching enables cleaner type handling. Together they improve readability, safety, maintainability, and compiler checking.

Feature	Purpose
record	Create clean immutable data objects (Holds data)
sealed	Control allowed subclasses (Restricts allowed types)
pattern matching	Process object types cleanly (Reads/handles types cleanly)

Together they make Java:

- cleaner
- safer
- less verbose
- easier to maintain

1. Records → Simple Data Containers

Before records:

```
class UPI {  
    private String upild;  
  
    constructor  
    getter  
    equals  
    hashCode  
    toString  
}
```

Too much boilerplate.

With Record

```
record UPI(String upild) {}  
record Card(String cardNumber) {}  
Very clean.
```

2. Sealed Classes → Restrict Allowed Types

Now business says:

Only UPI, Card, and Cash are valid payments.

```
sealed interface Payment  
    permits UPI, Card, Cash {}
```

Now nobody can create:

```
class BitcoinPayment implements Payment {}
```

3. Pattern Matching → Handle Types Cleanly

Now we process payments:

```
switch(payment) {  
  
    case UPI u ->  
        processUPI(u);  
  
    case Card c ->  
        processCard(c);  
  
    case Cash c ->  
        processCash(c);  
}
```

Java:

- checks type
- casts automatically
- gives typed variable

Very readable.

Why These Features Fit Together Perfectly

Step 1 — Records

Create clean immutable data objects.

Step 2 — Sealed Classes

Define all valid types safely.

Step 3 — Pattern Matching

Handle those types elegantly.

Race Condition Problem

27 May 2026 06:51

Absolutely.

Let's create a FULL runnable Java example that clearly demonstrates:

- what race condition is
- how multiple threads corrupt shared data
- why instance variables are dangerous
- how synchronization fixes it

This is one of the best beginner examples.

First Important Understanding

In Spring Boot:

@Service

creates:

Singleton Bean by default

Meaning:

ONLY ONE OBJECT

for entire application

Not:

one object per request

This is THE key.

What Does Singleton Mean?

Singleton means:

Only ONE object exists

for the entire application.

Not:

- one object per user
- one object per request
- one object per thread

ONLY ONE.

Dangerous Design

@Service

```
public class UserService {
```

```
    private User currentUser;
```

```
    public User getUser(int id) {
```

```
        currentUser = fetchFromDB(id);
```

```
        return currentUser;
```

```
    }
```

```
    private User fetchFromDB(int id) {
```

```

    return new User(id, "User-" + id);
}
}

```

What Beginners Imagine

Many freshers think:

User-1 → UserService Object-1

User-2 → UserService Object-2

User-3 → UserService Object-3

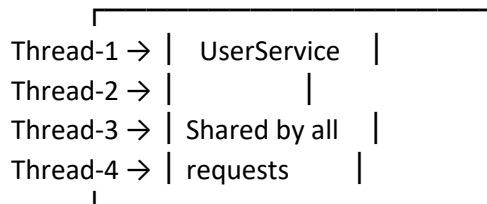
NO ✗

That is NOT how Spring works by default.

Actual Spring Behavior

Reality:

ONE OBJECT



All requests use SAME object.

This is THE key concept.

Step-by-Step Disaster

Thread-1

User requests:

GET /user/1

Executes:

currentUser = User(1)

But BEFORE returning response...

CPU switches thread.

Thread-2

Another request:

GET /user/2

Executes:

currentUser = User(2)

Now:

currentUser no longer User(1)

It became:

User(2)

Thread-1 Resumes

Now Thread-1 returns:

return currentUser;

But:

currentUser is now User(2)

So:

User-1 request accidentally gets User-2 data ✖

Huge production bug.

This Is Why Instance Variables Dangerous

Because:

private User currentUser;

is:

Shared across ALL requests

Visual Timeline

Thread-1:

currentUser = User(1)

CPU switch

Thread-2:

currentUser = User(2)

CPU switch

Thread-1 returns currentUser

→ returns User(2) ✖

Real Production Disaster

Possible issues:

- user sees another user's data
- payment corruption
- wrong OTP
- security breach

Very dangerous.

Stateless Means

Service should NOT store request-specific mutable data.

Good service:

@Service

public class UserService {

```

public User getUser(int id) {

    return fetchFromDB(id);
}
}

```

No shared mutable state.

Safe.

Real Enterprise REST API Rule

NEVER store:

- logged-in user
- request data
- transaction data
- OTP
- payment amount

inside singleton bean fields.

Instead Store In

- local variables
- database
- Redis
- request object
- JWT token

Example 1 — Race Condition Problem

Without Synchronization

```

class BankAccount {
    private int balance = 1000;
    public void withdraw(int amount) {
        if(balance >= amount) {
            System.out.println(Thread.currentThread().getName()
                + " is withdrawing ₹" + amount);
            // Simulate delay
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            balance = balance - amount;
            System.out.println(Thread.currentThread().getName()
                + " completed withdrawal.");
            System.out.println("Remaining Balance: ₹" + balance);
        }
        else {
            System.out.println(Thread.currentThread().getName()
                + " -> Insufficient Balance");
        }
    }
}

public class RaceConditionDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
    }
}

```



```

Thread user1 = new Thread(() -> {
    account.withdraw(800);
}, "ATM-User");
Thread user2 = new Thread(() -> {
    account.withdraw(500);
}, "NetBanking-User");
user1.start();
user2.start();
}
}

```

What You May See

Possible output:

ATM-User is withdrawing ₹800

NetBanking-User is withdrawing ₹500

ATM-User completed withdrawal.

Remaining Balance: ₹200

NetBanking-User completed withdrawal.

Remaining Balance: ₹500

Problem

Total withdrawn:

₹800 + ₹500 = ₹1300

from:

₹1000

Impossible in real banking.

This is:

Race Condition

Why It Happened

Because BOTH threads accessed:

private int balance
simultaneously.

Both saw:

balance = 1000

before update happened.

Visual Understanding

Thread-1 reads 1000

Thread-2 reads 1000

Thread-1 updates → 200

Thread-2 updates → 500

Final balance corrupted.

Now Let's FIX It

Example 2 — Using synchronized

```

class BankAccount {
    private int balance = 1000;
    public synchronized void withdraw(int amount) {

```

```

if(balance >= amount) {
    System.out.println(Thread.currentThread().getName()
        + " is withdrawing ₹" + amount);
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    balance = balance - amount;
    System.out.println(Thread.currentThread().getName()
        + " completed withdrawal.");
    System.out.println("Remaining Balance: ₹" + balance);
}
else {
    System.out.println(Thread.currentThread().getName()
        + " -> Insufficient Balance");
}
}
}

public class SynchronizedDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        Thread user1 = new Thread(() -> {
            account.withdraw(800);
        }, "ATM-User");
        Thread user2 = new Thread(() -> {
            account.withdraw(500);
        }, "NetBanking-User");
        user1.start();
        user2.start();
    }
}

```

Correct Output

ATM-User is withdrawing ₹800
 ATM-User completed withdrawal.
 Remaining Balance: ₹200
 NetBanking-User -> Insufficient Balance
 Correct behavior ☒

What synchronized Did

public synchronized void withdraw()
 means:

“Only ONE thread can execute this method at a time for this object.”

Important Understanding

Without synchronized:

Multiple threads enter simultaneously

With synchronized:

One thread enters

Others wait

Why Thread.sleep() Added?

Very important.

Without delay:

- race condition may happen too fast to notice

Sleep increases probability of thread collision.

It simulates:

- DB calls
- API calls
- network delay

Real backend systems constantly wait like this.

Real Spring Boot Connection

In Spring Boot:

@Service

class BankService

is singleton by default.

Meaning:

ONE object shared across all requests

So:

private int balance

becomes shared mutable state.

That's why race conditions happen in real applications too.

Mutable Shared Data Means:

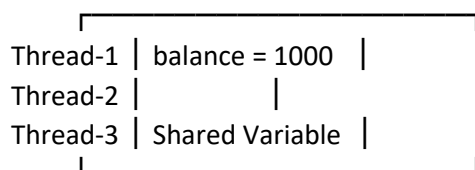
Data that:

- can change
- AND is accessed by multiple threads simultaneously

This is dangerous.

Visual Understanding

ONE OBJECT



Important Beginner Rule

Local Variable → Usually Safe

int x = 10;

Each thread gets separate copy.

Instance Variable → Dangerous

private int balance;

Shared across threads.

Interview-Friendly Summary

Race condition occurs when multiple threads simultaneously modify shared mutable state without proper synchronization, causing inconsistent or unpredictable results. It can be fixed

using synchronized, locks, atomic classes, or by avoiding shared mutable state.

Instance variables inside singleton Spring beans are shared across all concurrent requests. If mutable request-specific data is stored in them, multiple threads can overwrite each other's data causing race conditions and inconsistent responses. Local variables are safer because each thread gets its own separate copy in stack memory.

One-Line Golden Rule

Never store request/user-specific mutable data inside singleton Spring beans.

This one rule prevents MANY production concurrency bugs.

To Explore

26 May 2026 00:46

1. Why records should not be used. (what does mutable and immutable objects, get some enterprise example)

Common Mistakes

Using records for mutable objects

Bad fit for:

- JPA entities
- Hibernate entities

Because ORM frameworks expect mutability.

2. How come other user can use our code, can u give the example. (when not used sealed class)

1. Better Domain Modeling

Some real-world concepts are inherently limited. For example, a `PaymentMethod` might only ever be `CreditCard`, `PayPal`, or `BankTransfer`. Sealing ensures no rogue developer can randomly introduce a `MagicBeans` payment method into your system.

1. What is Package-private constructors? Used before sealed class.
2. What would be the problem.

Imagine if somebody could do:

```
class FakeString extends String
Disaster.
Java protects critical abstractions carefully.
```

3. why:
 - sealed classes
 - records
 - pattern matchingwere introduced together.
4. Can we have sealed interface also, ? Difference between sealed class and sealed interface.
5. What are the different possible deployment options we have VM, Docker etc? how it's different ?
6. How internally works., I can understand for single user hit and response but 10,000 users hit your API concurrently? How it works internally?

Suppose:

- 50,000 users checking balances simultaneously

Each request:

- calls DB
- waits for network
- waits for response

```

class BankAccount {

    private int balance = 1000;

    public void withdraw(int amount) {

        if(balance >= amount) {

            System.out.println(Thread.currentThread().getName()
                + " withdrawing " + amount);

            balance = balance - amount;

            System.out.println("Remaining Balance: " + balance);
        }
    }
}

```

7. I'm a backend application developer, I have done most project in CRUD application. What else I can try to upskill myself as java developer.
8. What is IO bound applications.
9. What Is Thread Pinning?
10. What is ractive programming.
11. Understand about HikariCP and connection Pooling in detail.
12. Why we should not instamce varoable for balance and what does it mean.
 - application may run on multiple servers
 - each server has separate memory

13. Explain this staement regarding rzce condition.

Suppose:

- 5 Spring Boot instances
- deployed behind load balancer

Now:

synchronized withdraw()
works only:

inside ONE server

But another server can still modify same account simultaneously.

That's why:

- DB locking
- transactions
- optimistic locking

become essential.

14. Explain this better.

```

10,000 Users
  ↓
Load Balancer
  ↓
10 Spring Boot Instances
  ↓
Tomcat Thread Pools

```

